PIVOT INDEX OF AN ARRAY

Finding the leftmost index where the sum of the elements on its left equals the sum of the elements on its right, using pointers in C

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that reads the size and the elements of a variable-length integer array arr[]. The array must be passed to a function named pivot_index() using its address, and pointers must be used to pass the array between the calling function and pivot_index().

The function pivot_index() must:

- Calculate the left sum (sum of all elements strictly to the left of each index).
- Calculate the right sum (sum of all elements strictly to the right of each index).
- Treat the left sum as 0 for the first element.
- Treat the right sum as 0 for the last element.
- Find and display the leftmost index at which left sum = right sum.
- Display -1 if no pivot index exists.

2. Objective

To understand and implement array traversal, dynamic memory allocation and pointer-based parameter passing in C, and to solve the pivot index problem in a single efficient pass over the array.

3. Concepts Used

- Arrays and pointer arithmetic (*(arr + i) is equivalent to arr[i]).
- Dynamic memory allocation using malloc() and free() for a variable-length array.
- Passing an array to a function by its base address.
- Loops (for) and decision making (if / else).
- Prefix-sum idea: right sum = total sum - left sum - current element.

4. Algorithm

1. Start the program.
2. Read the size n of the array and allocate memory for n integers using malloc().
3. Read the n elements into the array using a pointer.
4. Call pivot_index(arr, n), passing the base address of the array.
5. In pivot_index(), compute total = sum of all elements. Set left_sum = 0 and pivot = -1.
6. For each index i from 0 to n-1: calculate right_sum = total - left_sum - arr[i].
7. If left_sum is equal to right_sum, set pivot = i and stop (this is the leftmost pivot index).
8. Otherwise add arr[i] to left_sum and move to the next index.
9. If pivot is still -1, display -1; otherwise display "Pivot index: pivot".
10. Free the allocated memory and stop the program.

Note: For index 0 the left sum starts at 0, and for the last index the right sum automatically becomes 0 (because total - left_sum - arr[n-1] = 0 when nothing lies to its right). Time complexity is O(n) and extra space is O(1).

5. Source Code

File name: pivot_index.c

```
#include <stdio.h>
#include <stdlib.h>

/* Finds and displays the leftmost pivot index of the array.
   The array is received through a pointer (its base address). */
void pivot index(int *arr, int n)
{
    long total = 0, left sum = 0, right sum;
    int i, pivot = -1;

    /* Step 1: total sum of all elements */
    for (i = 0; i < n; i++)
        total += *(arr + i);

    /* Step 2: check every index from left to right */
    for (i = 0; i < n; i++)
    {
        /* right sum = total - left sum - current element */
        right_sum = total - left_sum - *(arr + i);

        if (left_sum == right_sum)
        {
            pivot = i;      /* leftmost pivot found */
            break;
        }
        left_sum += *(arr + i);   /* update left sum for next index */
    }

    if (pivot == -1)
        printf("-1\n");
    else
        printf("Pivot index: %d\n", pivot);
}

int main()
{
    int n, i;
    int *arr;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid array size.\n");
        return 1;
    }

    arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", arr + i);

    pivot_index(arr, n);   /* array passed using its address */

    free(arr);
    return 0;
}
```

6. Sample Input and Output

Example 1

```
Enter the size of the array: 6
Enter 6 elements: 1 7 3 6 5 6
Pivot index: 3
```

Explanation: Index 3 is a pivot index because the left sum (1 + 7 + 3 = 11) is equal to the right sum (5 + 6 = 11).

Example 2

```
Enter the size of the array: 3
Enter 3 elements: 1 2 3
-1
```

Explanation: There is no pivot index, as no left sum can be found to be equal to a right sum.

Example 3

```
Enter the size of the array: 3
Enter 3 elements: 2 1 -1
Pivot index: 0
```

Explanation: Index 0 is a pivot index because the left sum (0) is equal to the right sum (1 + (-1) = 0).

7. Dry Run (Example 1)

Array: 1 7 3 6 5 6, total sum = 28. For each index, right sum = 28 - left sum - arr[i].

Index (i)	arr[i]	Left Sum	Right Sum	Equal?
0	1	0	27	No
1	7	1	20	No
2	3	8	17	No
3	**6**	**11**	**11**	**Yes**

The loop stops at i = 3, so the pivot index is 3.

8. Additional Test Cases

Array	Size	Output
5	1	Pivot index: 0
0 0 0 0	4	Pivot index: 0
-1 -1 0 1 1	5	-1
2 3 -1 8 4	5	Pivot index: 3

9. Conclusion

The program successfully reads a variable-length array, passes it to pivot_index() using pointers, and finds the leftmost pivot index in a single pass using the relation right sum = total - left sum - current element. It produces correct results for all given examples as well as for edge cases such as a single-element array and an array of zeros.